

Introduction

C.V Jawahar

1.1 Introduction

Recent years had seen the success of AI in various walks of our life. It looks like it is going to affect in much deeper manner in the coming years. What is really happening? In the simplest form, we have found ways to come up with acceptable solutions for a large number of problems that could impact our day to day life. Why is it happening? There are many answers to this. In short, our algorithmic understanding on how to solve has improved. Our technology infrastructures (sensors, system logs, user logs, personal compute and storage availability etc.) has grown up a lot leading to the availability and accessibility of the data for research and development.

Today's AI is dominated by data driven techniques. A machine is able to see/observe how human (or an expert) is taking certain decisions in many many situations and this data/experience is used to build a solution that aims to mimic the human expertise. In this sense, many solutions that we see every day are the right examples of AI. Our mobile phones or special assistive gadgets is able to understand our sound, meaning and give you back a reply in a human like voice. Not just that we are hearing about the potential of machines that can drive cars on the streets safely like a human, also play even complex games with a human, and win comprehensively.

For many problems of interest, human error rates and machine error rates are coming closer and closer. This may not simply mean that machines are approaching the human capabilities (in all respects). This is limited success. However, we see that the situations where machines can help humans is systematically increasing. Every time more problems fall into this space, we celebrate the success of AI!!

For example, we would like the lights and fans at home to be controlled by algorithms. If we keep a simple timer and switch off and on, there is no practical intelligence. We could improve it by looking at some occupancy sensors. This makes better sense. We can also start looking at your regular usage patterns and do switch on a fan or an air conditioner to give you comfort before you reach. This becomes more complex. Indeed, such a prediction needs to use your past data (prior knowledge) and the evidences that I have now. (I see you started from your office.). We can ask the machines to be still smarter by economically optimizing the solution (how much we pay vs the risk of wasting the power vs risk of providing the comfort). We can also further advance the solution by looking at your face (tired, you need a coffee when you reach home), your email (you are free of the office pressure and let me recommend a nice channel to watch at home). As



Figure 1.1: Success Stories of modern AI: (a) Autonomous Cars (b) Alexa (c) AlphaGo

you can see most of these are not too far for the present day technology. What to observe at this stage is that (i) a wide spectrum of solutions are possible (ii) they need different skills, and process different data (ii) they need some of the very complex optimization to make the best decisions. We will see all these aspects again and again as we move forward.

1.2 Data and Representation

Let us consider a problem of classifying emails into "personal", "professional" or "promotional". This is a useful functionality to have in any email client. Some people may want a different classification like "family", "friends" and "fun". The problem may look very different on a quick look. However it is not. If we can develop an application/solution/code that can do the first from a number of examples, then the same solution will work for the second too. The main difference will be the data that is used to learn or train the solution. i.e., a learning algorithm that we may use for solving one problem may be very well useful for a wide variety of problems, even if these real world problems may look very different. This makes our life easy. We have only fewer learning problems to master than the real world problems that we may solve.

What is the data/experience in this case? We expect that some one gives us reasonably large number of emails and their corresponding labels. Here "personal", "professional" and "promotional" are labels. With out loss of generality, we will call them the class label y as 1, 2 or 3. This convinces us that if we move towards the second problem of "family", "friends" and "fun", there is no need to change the notations/solution. When your final application classifies an email as "1", we will have to interpret it as "personal" or "family", depending on convention we used. Machine does not really know what '1' means.

A very important task at this stage is to have a representation for the email that our algorithms can process. Emails could be of varying length. We, in general, prefer a representation that is of fixed length. We use the words "representation", "features" and "embedding" interchangeably. The last one (embedding) has become more popular in the recent years. For example, our first feature could be whether it comes from some one who is in our address book (a Boolean value Yes/No). Another potential feature could be whether the email has a word "buy" or not. We typically have many such features (say hundred of them i.e., $d=100$) to represent an email. We look at them as elements of a vector and treat the representation of email as $\mathbf{x} = [x^1, x^2, \dots, x^d]^T$.

Though 100s of features will not be a serious problem in computation while building the solution or deploying the solution, we are handicapped with our intuition/visualization skills 2D (or in the best case 3D). Therefore, for visualizations etc., we use only two features as shown in the Figure ???. Do not confuse this simple visualization with that of reasonably comprehensive representation of the real world objects/phenomena (such as email) with hundreds or even thousands of features. By the way, in today's ML, we are not much worried about such larger dimensional representation (though we may talk about dimensionality reduction in a later stage). What we may worry later is how complex is the concept, and not how many features or

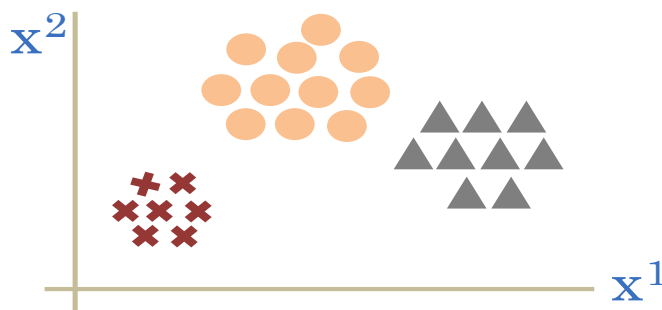


Figure 1.2: A two dimensional feature space. Samples can be visualized as points and the respective labels as the shape.

examples we have. (we will talk about the complexity of the concept and solution at a later stage).

Let us come back to Figure ???. We have three types of emails and they are shown with the help of two features x^1 and x^2 . Each point is a representation/visualization of the email. We have three types of emails, class 1, 2 or 3. They are shown as "circle", "cross" or "triangle". What else do we observe? All personal emails are "together" and similarly for "professional" and "promotional". This is nice. If a representation $[x^1, x^2]^T$ is doing that, we say that this representation is very good. Also if we want to separate them, it looks quite simple. The concept of a "personal email" is simple in this feature space or with this representation. Indeed, getting a useful representation is a big deal. We will see them later.

1.3 Example

Let us consider another example now. We are given fruits and we would like to classify them as either apples or oranges. (The problem of separating fruits and vegetables and even grading them is a very practical and important problem. We do not have hardware and software for this in many Indian farms and markets!!). Our objective is to take this as a toy text book problem at this stage. We assume that each fruit is characterized by two features (i) colour and (ii) density. They could be captured with certain sensors and represented as a real number in a fixed range. (As discussed in the last section, such simplified representations may not be enough for practical purposes. But that is enough at this moment for our explanation.). Let there be N such fruits $\mathbf{x}_1, \dots, \mathbf{x}_N$ where each fruit is represented with two features as $\mathbf{x} = [x^1, x^2]^T$. This data is shown in Figure 4.10.

Separating apples from oranges is that of finding a line $w_1x^1 + w_2x^2 + w^3 = 0$ such that apples are on one side of the line and oranges are on other side of the line. Mathematically, apples have $w_1x^1 + w_2x^2 + w^3 < 0$ and oranges have $w_1x^1 + w_2x^2 + w^3 \geq 0$. (When it is equal to zero, the decision could have been either side. We do not care seriously.)

This raises a set of important questions:

- *Who gives the coefficients w_1, w_2 and w_3 in this case?* Ans: That is our major problem. We will see many methods to do this in later chapters. We learn these coefficients from the examples that are given to us.
- *Is it possible to have a line like this always?* Ans: No. We wish such a line exists. In general, we keep our objective as finding the best line. (i.e., the line that makes minimal errors).
- *Why did we limit to line?* Ans: There is no real need to limit to a linear model like line here. We can consider nonlinear ones, such as parabolas or other conics. It can also be far more complex functions.

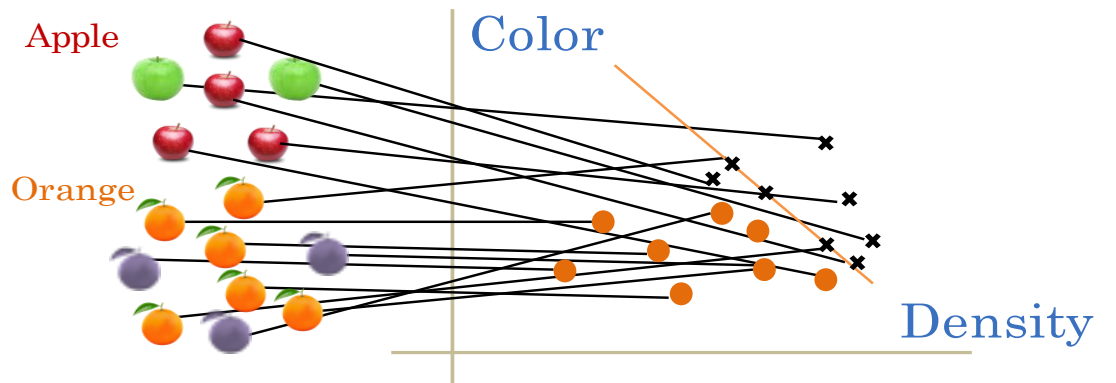


Figure 1.3: We can represent real world objects (like fruits) as points in a feature space. We would like to find lines/planes/hyperplanes that can separate apples from oranges.

That is what deep neural networks are.

- *Is it that we always have only two classes?* No. Many real world problems have more than two classes. We can build multi-class classifiers from binary (two class) classifiers. There are also many natural multi class classifiers.

We know that a line of the form $w_1x^1 + w_2x^2 = 0$ passes through the origin always. If you want to consider all potential lines, we would like to model this as $w_1x^1 + w_2x^2 + w^3 = 0$. In practice this is seen as $\mathbf{w}^T \mathbf{x}$ where $\mathbf{w} = [w_1, w_2, w_3]^T$ and $\mathbf{x} = [x^1, x^2, 1]^T$. i.e., $\mathbf{x}$ is augmented with a 1 at the end. And corresponding w^3 is called the bias term. In most places, we may not be explicitly mentioning the bias term. But it is assumed.

We can now think of a line that separates personal emails from the other two classes. Similarly we can think of two other lines/classifiers too. Such a solution is an acceptable starting point. But not perfect. For example, the first one says the email as personal, second one says it as professional and the third one says it as promotional. How do we decide now? (Think of an intuitive solution to address this.)

1.4 Summary

Before we end this chapter, let us first summarize and then introduce some of the terminologies that we may see later.

We are given some real world objects and the corresponding labels. We, in general, represent it as $(\mathbf{x}, y)$. Often $\mathbf{x}$ is a vector of d features and y is an integer or a class label. Our problem is to find a function $f(\mathbf{x})$ which is parameterized by $\mathbf{w}$. In the simplest form $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$

Training is the process of building the model or finding the appropriate $\mathbf{w}$ given the data. Testing is the process of using the learned model/function and evaluating on a novel sample. i.e., use of a learned classifier on a new email that we received now. In general, training is compute intensive, while testing is much simpler. In many practical situations, testing time is more important than the training time.

The examples that we saw were mainly supervised. In this case, we are given $\mathbf{x}$ and y . Here y is the supervisory signal. Such problems are often called supervised learning, Most of the successful examples at this stage are supervised.

ML and Learnable Functions

C.V Jawahar

Let us repeat our understanding of the problem. We are given a number of examples $(\mathbf{x}_i, y_i)$. Our interest is mostly in the supervised learning task, where y_i is available for each $\mathbf{x}_i$. We will see some other forms of learning also at later stages.

We are interested in learning a function $f()$ that can relate $\mathbf{x}$ and y i.e., $y_i = f(\mathbf{x}_i)$. Though we wish y_i to be equal to $f(\mathbf{x}_i)$ for all i , it is not practically possible to have the equality for all the samples at the same time. We have to constrain our requirement to only approximately equal. To be more correct, we will keep our goal as something like “minimize the discrepancy between y_i and $f(\mathbf{x}_i)$ ”. Often this discrepancy is what is called the loss or error. We are interested in minimizing this error. That is the major step in learning.

Let us assume that $\mathbf{x}$ is an email represented using two features x^1 and x^2 . Note that in practice two features/measurements are not enough for an email to be represented. But this makes our discussion simple here, with no loss in generality. Let us assume we are interested in classifying the email as ‘spam’ or ‘not-spam’ (a binary classification problem). This corresponds to y being 0 or 1. (indeed, we can also use the notations like -1 and $+1$ for the two classes. It is only a matter of convenience, what notation we use.)

How do we learn a function? Let us assumed that the function is a linear function, i.e., $y = w_1x^1 + w_2x^2 + w_3$. Then learning the function is same as finding the “best” w_1, w_2 and w_3 . One brute force way of finding the $\mathbf{w} = [w_1, w_2, w_3]^T$ is by trying out all possible values. That is simply impractical. Since $\mathbf{w}$ is a vector of real numbers, there are infinite possibilities. (For example, we are interested in one of the infinite many possible lines in 2D).

Assume $\mathbf{x}$ was the representation of emails, one of the possible $\mathbf{w}$ will be the best to separate spam emails from the non-spam emails. Another line will separate personal emails from other emails. This is interesting. With a “minor change” (i.e., $\mathbf{w}$ ’s change), we obtain very different functionality (i.e., spam vs non-spam or personal vs other emails). How did we achieve this? Data allowed us to learn such widely different functionality with minimal effort. Now data is able to effectively create/suggest functions that can solve widely different problems. (The same machine learning algorithm is useful for learning wide variety of functions). Do appreciate the major difference now. Our ML algorithms are now writing functions that can solve very complex tasks. They output learned functions that can be plugged into some of the larger software systems. The same ML algorithm creates different functions and help in solving different problems.

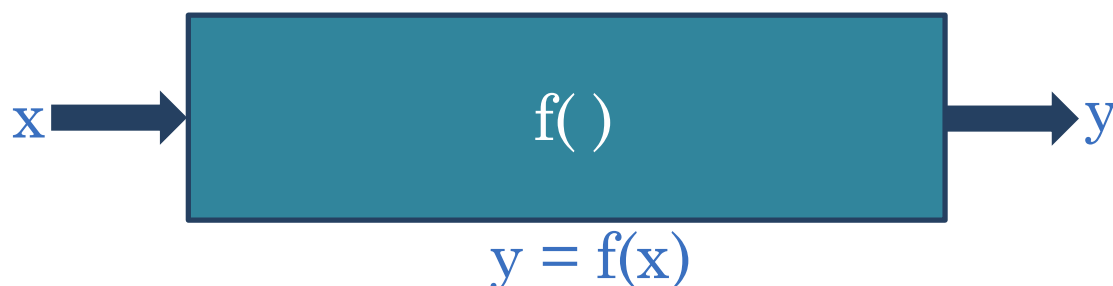


Figure 2.4: Data Driven Functions. Given data in the form of $(\mathbf{x}_i, y_i)$, we can come up with function $f()$ that can predict y for a new $\mathbf{x}$



Figure 2.5: A Simple Data Driven Functions. A linear classifier classifies the sample as label 1 or 0 with a simple function. With a change in the coefficients, the same lines of code can solve many different problems.

Why do we have to limit to linear functions? There is no need. The function can be very complex. It can be as complex that we can not (or we do not want to) write it on a paper/board. This is what many of the modern machine learning algorithms (such as deep neural networks) are.

2.1 Examples

Example 1 Let us consider our text book problem of converting Fahrenheit to Celsius. A textbook problem that we solve while learning computer programming. We can solve this problem in two different ways (i) We can use the model/equation $C = (F - 32) * \frac{5}{9}$ or $C = 0.55F - 17.77$. (ii) We can have a look up table of all the Celsius (say from -273 to 1000) in a look up table and use it directly whenever we want. The solutions of the second style is memory intensive. They may not require much computation. But they are limited to only the integer values we stored in the table (it does not tell us anything beyond the limits or with fractional values in between). The first one is attractive. But it requires us to know the relationship $C = f(F)$ ahead of time. That is not possible in most practical situation. We may not know how the phenomena works in practice. (Do we know what we understand when certain sound signals reach our ear?).

The data driven functions move from (ii) to (i). We start with a number of examples (no need of all the examples). We learn the function and then build the capability to predict for any of the future examples.

Example 2 Consider the example of classifying a fruit as orange “if $2x^1 - 3x^2 + 3 \geq 0$ and apple otherwise”. We already argued that the data gave us the coefficients $\mathbf{w} = [2, -3, 3]^T$. This simply mean that data has written a simple function like in Figure 2.5. A simple 4 line function has the knowledge of many (may be even 1 Million) examples. A very compact representation of the knowledge that we captured from the examples to solve a problem is represented in 4 lines. This is impressive. A dot product in d dimension requires only d multiplications and $d - 1$ additions. Even with d becoming few hundreds or thousands, this can be computed efficiently.

In most practical situations, such data driven functions are used only at the test time. They are simple. They encode the knowledge from the data/examples.

2.2 Softwares and Systems

Before we end this chapter, there is an important distinction to note. In the traditional coding, we worry about the efficiency of the solution. We want the code to yield the results as fast as possible. In our case,

it is not it is not exactly so. We want the most accurate solutions, even if we spend a little extra time in computing $\mathbf{w}$. We worry about the efficiency of the learned function $f()$, and not about the training code. But not about how fast learning algorithm yield a solution.

In traditional setting we worry about ‘bugs’ in the code. Bugs lead to errors. Here our function can make wrong prediction. This is natural. Code is not buggy. Algorithm is not faulty. It is the accuracy/performance that we can obtain. Note that we are predicting the performance of a solution in the future situations (that we have no knowledge at this stage). Then there is all chance that the predictions can go wrong. However, if our model is good, our accuracy can be high. However, there is no guarantee that it is 100% accurate. An algorithm for sorting an array will always correctly sort. However an algorithm for predicting an email as spam need not predict it correct always. My definition of a spam email could be different from that of yours. My definition of a spam email today can be very different from my definition of spam email 10 years back.

We saw that a simple function embeds knowledge of Millions of examples. This makes the life of maintenance and modification of the code very difficult tomorrow. This data driven function does not say why these coefficients were used ? (other than saying that these are the best coefficients). This makes the solution difficult to explain. (In the linear case we discussed, we can make some explanation. However, if the model becomes more and more nonlinear and non parametric, it becomes practically impossible to explain.) In cases, where the explanation is very important (for legal reasons, for customer satisfaction, for fairness in the decision making etc.) this is a disadvantage. Also over years, the maintenance and improvements of such data driven schemes will lead to a new set of problems.

Figure 2.6 shows a software system that has a specific function that is learned. Note that the data is used to learn this function offline (training phase) and the trained function alone can be part of the large software system. Most of our interest is in the offline phase of creating such functions.

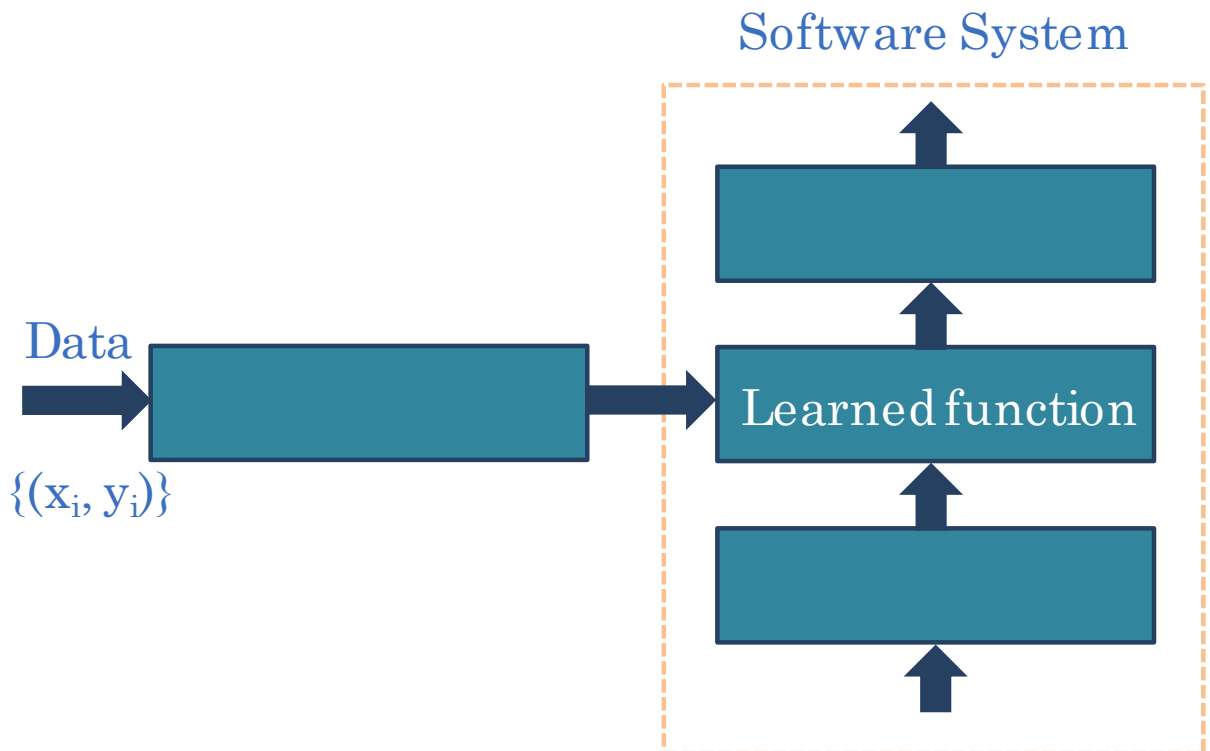


Figure 2.6: Data Driven Functions in large software systems

Supervised Learning

C.V Jawahar

3.1 Problem Space

Let us start with our basic problem again. We are given a number of examples (data) in the form $\{(\mathbf{x}_i, y_i)\}$ for $i = 1, \dots, N$. For the simplicity, we assume that $\mathbf{x}_i$ is a real d -dimensional vector. And y_i is a scalar (say real number or an integer). Our interest is in finding a function $f()$ such that $f(\mathbf{x}_i)$ is same (or very similar) as y_i .

At this moment, we will deal with the requirement of the “very similar” to be as close as possible for all the N training samples that we have. Later we should also make sure that the same function will work for all the samples that we come across in the future also (i.e., unseen samples). That is when we can really say that we have learned the function.

For example, $\mathbf{x}_i$ could be an email that is represented with a set of features, and y_i could be 0 or 1 corresponding to “spam” (1) or “not spam”(0). In this case it is a classification problem. One may also formulate the problem as classification of the email into “spam” (0), “personal” (1) and “professional” (2). In this case, this is a multiclass classification problem. (There are three classes here.) We can also attempt to predict “how important/urgent” is an email by looking at $\mathbf{x}$. In this case, then y_i is a real number (say in the range $[0 - 1]$ (i.e., 0 to 1) where 0 means least urgent and 1 means extremely urgent.

Note that in all the above example situations, we are given many examples and our objective is to learn the function $f(\mathbf{x})$ that works well (or meets the expectation) on the existing and the future situations. It is indeed impossible to have the future situations in our hand while doing the computations. Therefore, we have to work with the existing data and “hope” that it works well in the future too. (Not that bad. We have some formal tricks to make sure that it happens. Wait and watch.)

3.1.1 Classification

In classification problems y_i is an integer. What value we assign is of not much significance. For example, some people use 0 and 1, while some where else we can use -1 and $+1$ as the class IDs. The choice is often based on some conveniences (simpler form of equations!!). In both these cases, it is a “binary” classification.

Multi class classification where the number of classes is more than 2 is a popular case. Though multiclass classification is very popular and important in practice, many discussions in the linear classifiers assume that the number of classes is only 2. That makes the life simple. Many algorithms are inherently binary. But we can adapt them to multiclass situations.

3.1.2 Regression

In the case of regression, y_i is real quantity. It could be a real vector or a simple real number. Let us assume it as a real number at this stage. Though classification and regression may look very different on a quick look, most of the discussions we have on one is equally true for the other. With some minor changes, we can adapt an algorithm of one for the other.

3.1.3 Others

There are many other situations where the space of y has a structure. For example, y is a graph or a string. Such problems are beyond our interest at this stage. In general, such problems fall under the category of “structured prediction”.

3.2 How do we formulate ?

Now let us come back to our problem. We are given examples $\{(\mathbf{x}_i, y_i)\}$ $i = 1, \dots, N$. We first impose a structure/form for the function. Let us assume that the function is of the form $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$. (We will see more about such functions later.) Here the vector $\mathbf{w}$ has unknowns and really speaking, finding the function now means, finding the best (or most appropriate) $\mathbf{w}$.

Indeed we may not find a separate $\mathbf{w}$ that can make $y_i = \mathbf{w}^T \mathbf{x}$ for each i . We would like to have a single function (or single $\mathbf{w}$) that works for all the data. We also would not want to insist on the equality. (We want to really say $y_i \sim \mathbf{w}^T \mathbf{x}$) This could be due to various reasons including errors, noise or uncertainty in the data. Therefore we want to model the problem as that of finding the “most appropriate” $\mathbf{w}$ given the data/examples. This naturally lead to an optimization problem. Most appropriate in what sense? We need to define an appropriate sense or objective that can be computed. This is the objective function. In machine learning, we also use the term loss function or error function frequently. All these are used with very similar meaning.

Our problem is then to find $\mathbf{w}$ that minimizes

$$TotalLoss = \sum_{i=1}^N LossPerSample = \sum_{i=1}^N L(\mathbf{w}, x_i, y_i)$$

Why do we have to sum over i ? Why not products? That is also possible. We do summation so that differentiation is easier later. (Do you remember how to differential $u + v$ and uv ? If the objective is product of terms, we can take log and make it summation.) There may be many different ways in which you can define the loss functions.

3.3 Loss Function

Consider the regression problem in 1D. You have (x_i, y_i) . By augmenting 1, x_i becomes a 2 dimensional vector $\mathbf{x}_i = [x_i, 1]^T$. Our problem is to find the vector $\mathbf{w} = [w_1, w_0]^T$ such that the model is $y_i = w_1 x_i + w_0$. Look at this as a line fitting.

Error or loss in this case is the difference between the model prediction and actual.

$$L(\mathbf{w}, \mathbf{x}_i, y_i) = (y_i - \mathbf{w}^T \mathbf{x}_i)^2$$

We square the error such that loss is not negative. This is required since we will now be adding the loss from different examples to get the total loss.

And the total loss or objective is then sum over all the samples as:

$$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 \quad (3.1)$$

Note that we have only one $\mathbf{w}$ for all the samples.

Similarly for a classification problem, loss can be 1 if the classification is wrong and 0 if the classification is correct.

Let us assume that the classifier is

$$f(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} > 0 \\ -1 & \text{else} \end{cases} \quad (3.2)$$

$$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (1 - y_i \cdot f(\mathbf{x}_i)) \quad (3.3)$$

We will see later that this loss is not very attractive, since it is not differentiable.

3.4 Optimization

The optimization problem we need to solve is

$$\min_{\mathbf{w}} J(\mathbf{w}) \quad (3.4)$$

Note that after the optimization, we are more interested in the $\mathbf{w}$ that gives the minima than the value of the minima of the $J()$.

Though we know what problem to solve, very often we can not find the “best $\mathbf{w}$ ” in practice. The best $\mathbf{w}$ is represented as $\mathbf{w}^*$

There are two prominent classes of optimization problems:

- Convex optimization: A well behaved class of problem. You can find the optima efficiently. Provable guarantees exists.
- Non-Convex optimization problem: This is a class of nasty optimization problems. Unfortunately, we will encounter them very frequently. In this case, very often, we have to be happy with “a” minima/solution and not the best solution.

The difference between convex and non-convex objective functions are shown In Figure ??

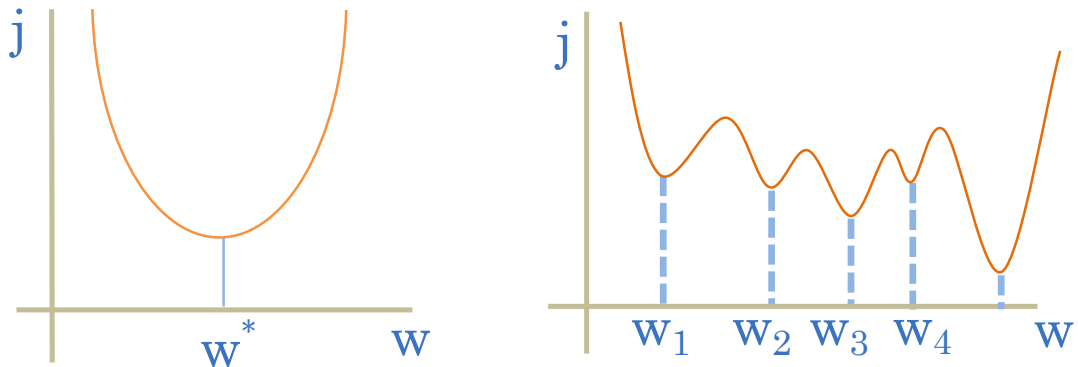


Figure 3.7: Examples of convex and non-convex objective functions. Convex has only one minima here. While the non-convex has many. All w_1 , w_2 , w_3 and w_4 are minimas. More precisely local minimas.